Rishtafy — Product Requirements Document (PRD)

Version: 1.1 *(Updated — Sections 7, 8, Appendix A revised)* **Phase:** 2 · Logic Doc 1 of 2
Project Type: University Semester Project (SE Course)
Team: Syed Shazan Ali Rizvi (23k-0847) · Muhammad Taaha (23k-0548) · Ebrahim Bin Moin (23k-0540) · Syed Hasan Haider (23k-0680)
Target Build Time: 2–3 weeks
Stack: Frontend — React (designed in Figma + Google Stitch) · Backend — Express JS · Database & Storage — Supabase

---

Table of Contents

1. Product Overview
2. Scope & Out of Scope
3. User Personas
4. User Stories
5. Feature List & Priority
6. Screen-by-Screen Flow
7. Success Metrics
8. Constraints & Assumptions

---

1. Product Overview

1.1 What is Rishtafy?

Rishtafy is an **intent-based matrimonial web application** for Pakistani/South Asian Muslim users. It connects singles seeking marriage in a verified, privacy-first environment, and uniquely supports a **Guardian/Parent role** so families can co-manage the search alongside the candidate.

1.2 The Core Problem Being Solved

> "Finding a spouse in Pakistan is either offline (rishta aunties, family networks) or done through culturally misaligned apps (Tinder, Muzz) with no verification and no guardian support. Rishtafy bridges this gap."

1.3 One-Line Value Proposition

Verified profiles. Guardian controls. Intent-only matching.

1.4 Semester Project Goal

Build a **functional MVP** that demonstrates the following three core flows end-to-end:

1. A **Candidate** can register, create a profile, search for matches, and send an Interest Request.
2. A **Guardian** can create/manage a profile on behalf of a candidate and review incoming requests.
3. An **Admin** can view all users, verify profiles, and manage the system from a dashboard.

The MVP does **not** need to be production-ready. It needs to be **demo-ready** — all three user flows must work without crashing during a live presentation.

---

2. Scope & Out of Scope

✅ IN SCOPE (Build This)

Feature	Why It's In
User registration & login (Candidate + Guardian + Admin)	Core auth, needed for everything else
Profile creation (bio, education, preferences)	The foundational product unit
Smart search & filtering	Core discovery mechanism
Interest Request system (send/accept/decline)	The primary user transaction
Chat (unlock only after mutual acceptance)	Demonstrates the intent-based model
Photo privacy (blur until request accepted)	Key UVP differentiator
Guardian dashboard (view/manage candidate profile + screen requests)	The unique feature no competitor has
Admin dashboard (user list, basic verification flag, stats)	Required for demo
Basic profile verification flag (manual, Admin sets it)	Simulates moderation without automation

❌ OUT OF SCOPE (Do NOT Build This Sprint)

Feature	Reason Excluded
AI Compatibility Scorer / Psychometric quiz	Complex ML pipeline; 2-week scope risk
Payment / Subscription billing	Stripe integration is a project in itself
Push notifications (mobile)	Not needed for web MVP demo
Video/Voice calling	Twilio/WebRTC complexity out of scope
ID document upload & automated OCR verification	Needs third-party API + compliance
Mobile native app (iOS/Android)	Web-responsive is sufficient for demo
Email OTP / SMS OTP at registration	Simplify to email/password auth for MVP

---

3. User Personas

Persona 1: The Candidate — "Zainab, 26"

- **Background:** Masters student at NED University, Karachi. Tech-savvy, uses Instagram daily.
- **Goal:** Find a compatible spouse who shares her values — without compromising her privacy or having to use a "dating app" that feels culturally wrong.
- **Pain Points:**
  - Doesn't want her photos visible to just anyone.
  - Previous apps felt unsafe due to fake profiles.
  - Wants her parents somewhat involved but also wants her own agency.
- **Behaviours:**
  - Browses profiles on her phone during evenings.
  - Sends 2–3 Interest Requests per week.
  - Would only chat with someone after mutual acceptance.
- **What she needs from Rishtafy:**
  - A verified, clean profile listing.
  - Photo-blur control.

- Clear status tracking on sent requests.

Persona 2: The Guardian — "Ammi (Mrs. Siddiqui), 52"

- **Background:** Homemaker in Lahore. Uses WhatsApp but not tech-savvy beyond messaging apps.
- **Goal:** Find a good proposal for her son, screen out unsuitable requests, and feel in control of the process.
- **Pain Points:**
  - Doesn't trust apps because "anyone could be on there."
  - Wants to see incoming proposals before her son does.
  - Finds most digital platforms confusing.
- **Behaviours:**
  - Checks the app once or twice a day.
  - Wants a simple inbox of incoming proposals to approve or reject.
  - Would call her son after approving a match.
- **What she needs from Rishtafy:**
  - A simple, clean Guardian Dashboard.
  - Clear "Approve / Decline" buttons on incoming Interest Requests.
  - Ability to manage her son's profile details.

Persona 3: The Admin — "Moderator Hassan"

- **Background:** Internal team member (for the demo, this is the developer/TA playing the admin role).
- **Goal:** Keep the platform trustworthy by verifying profiles and handling flagged accounts.
- **What he needs from Rishtafy:**
  - A simple table view of all registered users.
  - Ability to mark a user as "Verified."
  - Ability to deactivate/suspend an account.
  - Basic platform stats (total users, total requests sent, active chats).

---

4. User Stories

4.1 Authentication & Onboarding

ID	User Story	Acceptance Criteria
US-01	As a **Candidate**, I want to register with my name, email, password, and date of birth so I can create an account.	Registration form validates all fields. Duplicate email shows error. Successful registration redirects to profile setup.
US-02	As a **Guardian**, I want to register and select the "Guardian" role so I can manage a profile on behalf of a candidate.	Role selector present on registration. Guardian account creation succeeds and redirects to Guardian Dashboard.
US-03	As any user, I want to log in with my email and password so I can access my account.	Correct credentials → redirect to respective dashboard. Wrong credentials → clear error message.
US-04	As any user, I want to log out so my session is ended securely.	Logout button clears session. Navigating back does not show authenticated pages.

4.2 Profile Management

ID	User Story	Acceptance Criteria
US-05	As a **Candidate**, I want to fill out my profile (name, age, city, education, career, religious values, about me) so others can learn about me.	All fields save correctly. Incomplete profiles show a completion indicator.
US-06	As a **Candidate**, I want to upload a profile photo so others can see what I look like.	Photo uploads and displays on profile. Photo is blurred by default on search results.
US-07	As a **Candidate**, I want to set my partner preferences (age range, city, education level, profession) so I only see relevant matches.	Saved preferences correctly filter search results.
US-08	As a **Candidate**, I want to control my photo visibility so that my photo is only revealed when I accept an Interest Request.	Photo remains blurred for users who haven't had their request accepted. Photo is unblurred only for mutually accepted connections.
US-09	As a **Guardian**, I want to create and edit the candidate's profile on their behalf so I can	Guardian can edit all profile fields for the linked candidate. Changes are reflected immediately

ID	User Story	Acceptance Criteria
	manage their search.	on the candidate's profile.

4.3 Search & Discovery

ID	User Story	Acceptance Criteria
US-10	As a **Candidate**, I want to search for potential matches so I can find someone compatible.	Search results display as profile cards with blurred photos, name, age, city, education.
US-11	As a **Candidate**, I want to filter search results by age range, city, and education level so I can narrow down my options.	Applying filters updates the results list in real time (or on submit). Filters are persistent during the session.
US-12	As a **Candidate**, I want to view a full profile page for a potential match so I can learn more about them before sending a request.	Profile detail page shows all public fields. Photo is blurred if no accepted connection exists. A "Send Interest" button is present.

4.4 Interest Request System

ID	User Story	Acceptance Criteria
US-13	As a **Candidate**, I want to send an Interest Request to a profile I like so I can express my intent.	"Send Interest" button on profile. Request status changes to "Pending." User cannot send duplicate requests to the same profile.
US-14	As a **Candidate**, I want to see all my sent and received Interest Requests and their statuses so I can track my activity.	Requests page shows two tabs: "Sent" and "Received." Each request shows status: Pending / Accepted / Declined.
US-15	As a **Candidate (receiver)**, I want to accept or decline an Interest Request so I can control who I connect with.	Accept/Decline buttons present on received requests. Accepting unlocks chat for both parties. Declining removes the request from the active queue.
US-16	As a **Guardian**, I want to see all incoming Interest Requests for the candidate I manage, and approve or decline them, before the candidate sees them.	Guardian's inbox shows all incoming requests. Approve/Decline buttons present. Only approved requests become visible on the Candidate's "Received" tab.

4.5 Chat

ID	User Story	Acceptance Criteria
US-17	As a **Candidate**, I want to chat with a match only after we've mutually accepted each other's Interest Request, so that conversations are always intentional.	Chat interface is locked/inaccessible until both parties have accepted. Accepted matches appear in a "My Connections" list with a "Chat" button.
US-18	As a **Candidate**, I want to send and receive text messages in a chat so we can get to know each other.	Messages send and appear in real time (or near-real-time with a page refresh for MVP). Message history persists during the session.

4.6 Admin

ID	User Story	Acceptance Criteria
US-19	As an **Admin**, I want to view a list of all registered users so I can manage the platform.	Admin dashboard shows a table: name, email, role, registration date, verification status.
US-20	As an **Admin**, I want to mark a user's profile as "Verified" so that other users know this profile is genuine.	Admin can toggle a "Verified" badge on any user. Verified badge appears on the user's profile card in search results.
US-21	As an **Admin**, I want to deactivate a user account so I can remove bad actors from the platform.	Deactivated users cannot log in. Their profiles do not appear in search results.
US-22	As an **Admin**, I want to see basic platform stats (total users, total Interest Requests sent, active chats) on my dashboard so I can monitor platform health.	Stats are displayed as summary cards on the Admin dashboard homepage.

5. Feature List & Priority

> **Priority Legend:**
> 🔴 **P0 — Must Have** (Demo breaks without it)

🟡 **P1 — Should Have** (Core UVP, complete the story)
🟢 **P2 — Nice to Have** (If time allows in Week 3)

#	Feature	Priority	Notes
1	User Registration (Candidate + Guardian + Admin roles)	🔴 P0	Foundation of everything
2	Login / Logout / Session management	🔴 P0	
3	Candidate profile creation & edit	🔴 P0	
4	Profile photo upload	🔴 P0	
5	Search with basic filters (age, city, education)	🔴 P0	
6	View full profile detail page	🔴 P0	
7	Send Interest Request	🔴 P0	
8	Accept / Decline Interest Request (Candidate)	🔴 P0	
9	Chat unlock after mutual acceptance	🔴 P0	
10	Basic real-time or refresh-based chat	🔴 P0	
11	Guardian role — create/edit candidate profile	🟡 P1	Key differentiator
12	Guardian dashboard — approve/decline incoming requests	🟡 P1	Key differentiator
13	Photo-blur until accepted connection	🟡 P1	Key UVP feature
14	Admin dashboard — user list, verify, deactivate	🟡 P1	Needed for demo
15	Admin stats cards	🟡 P1	Nice for presentation slide
16	"Sent/Received Requests" tracking page for Candidate	🟡 P1	Completes the request flow
17	Verified badge on profile cards	🟢 P2	Visual trust signal
18	Partner preference saving & filter persistence	🟢 P2	Enhances search UX
19	Profile completion progress indicator	🟢 P2	UX polish
20	Responsive mobile layout	🟢 P2	Good to have for demo on phone

6. Screen-by-Screen Flow

6.1 Screen List

```
/                          → Landing Page (about Rishtafy + CTA to Register/Login)
/register                  → Registration page (name, email, password, role
selector)
/login                     → Login page

--- CANDIDATE SCREENS ---
/dashboard                 → Candidate home: quick stats, recent activity, nav links
/profile/edit              → Edit own profile (bio, education, preferences, photo)
/search                    → Search & filter page → shows profile cards
/profile/:userId           → View another user's full profile
/requests                  → Sent + Received requests (tabbed)
/connections               → List of accepted connections → chat entry point
/chat/:connectionId        → Chat screen with a specific accepted connection

--- GUARDIAN SCREENS ---
/guardian/dashboard        → Guardian home: manage candidate profile + request inbox
/guardian/profile/edit     → Edit candidate profile on their behalf
/guardian/requests         → Incoming requests to approve/decline

--- ADMIN SCREENS ---
/admin/dashboard           → Admin home: stats cards
/admin/users               → User list table: verify / deactivate controls
```

6.2 Critical User Flow: Candidate Sends & Receives a Match

```
[Zainab logs in]
    → /dashboard
    → Clicks "Find Matches" → /search
    → Applies filters (age: 26–32, city: Karachi)
    → Sees profile cards (photos blurred)
    → Clicks a card → /profile/user123
    → Reads bio, education, values
    → Clicks "Send Interest"
    → Status shows "Pending" on her /requests page
```

```
[Other party (Ahmed) logs in]
    → Sees new request on /requests (Received tab)
    → Clicks "Accept"
    → Both Zainab and Ahmed now appear in each other's /connections
    → Photos are now unblurred for both
    → Both can click "Chat" → /chat/conn456
    → They exchange messages
```

6.3 Critical User Flow: Guardian Screens a Request

```
[Mrs. Siddiqui (Guardian) logs in]
    → /guardian/dashboard
    → Sees "3 new Interest Requests for [Candidate: Ali]"
    → Clicks into /guardian/requests
    → Reviews first request (profile, bio, education visible)
    → Clicks "Decline" → request removed from queue, NOT sent to Ali
    → Reviews second request → Clicks "Approve"
    → Request now appears in Ali's (/candidate) Received tab
    → Ali can now Accept or Decline from his own dashboard
```

---

7. Success Metrics

> These define "done" for the semester project. If all of these pass, the project is demo-ready.

Metric	Target	How to Measure
Onboarding time	A new Candidate completes registration + basic profile in under 3 minutes	Time a team member doing a cold run
Search-to-request flow	User can find a match and send an Interest Request in under 5 clicks from the dashboard	Count clicks in a demo walkthrough
Guardian request flow	Guardian can approve or decline an incoming request in under 2 clicks from their dashboard	Live demo test
Chat unlock	After mutual acceptance, chat is accessible without any page errors	Manual QA test with two test accounts
Admin verify	Admin can mark a user as Verified and the badge appears on their profile within 1 page refresh	Manual QA
Zero crashes on demo day	All P0 and P1 flows complete without a 500 error or broken UI during the 15-minute demo	Full dry-run rehearsal the day before
Photo privacy	A non-connected user cannot see a candidate's real photo (blurred/hidden)	Test with two separate logged-in sessions
Role-based routing	After login, each role (Candidate / Guardian / Admin) is automatically redirected to its correct dashboard — no manual navigation required	Test login with one account of each role
Correct role enforcement	A Candidate cannot access `/guardian/*` or `/admin/*` routes, and a Guardian cannot access `/admin/*` routes — all return a redirect or 403	Manually attempt route access with wrong-role session
Photo upload success	A Candidate can upload a profile photo and it renders correctly on their profile card and detail page without a broken image link	Upload a photo, navigate to `/search` and `/profile/:userId` and confirm display
Guardian–Candidate link integrity	An Interest Request sent to a Guardian-managed Candidate appears first in the Guardian's inbox, not the Candidate's — and only reaches the Candidate after Guardian approval	End-to-end test using three accounts: Sender, Guardian, Candidate

Metric	Target	How to Measure
Design–Figma consistency	All implemented screens visually match the Figma prototype for layout, component structure, colour, and typography to a degree that a first-time viewer would not notice a difference	Side-by-side review of each screen against the Figma file before demo day
Supabase data persistence	All data (profiles, requests, messages, connections) survives a full browser refresh and a server restart — nothing is stored in-memory only	Restart the Express server mid-session and confirm data is intact

8. Constraints & Assumptions

8.1 Technical Constraints

- **Build time:** 2–3 weeks. No feature should take more than 3 days to implement.
- **Team size:** 4 people. Divide: 1 backend (Express JS), 1 frontend (React), 1 full-stack support, 1 QA/integration.
- **Frontend framework: React**. Component structure should mirror the Figma design system. Use functional components and React Router for client-side routing.
- **Backend framework: Express JS**. RESTful API design. Each route group maps to a role (`/api/candidate/`, `/api/guardian/`, `/api/admin/`). The Express server connects to Supabase via the Supabase JS client or REST API.
- **Database & Auth: Supabase**. Use Supabase Auth for user authentication (email + password). All application tables (profiles, interest_requests, connections, messages, guardian_links) are created in the Supabase Postgres instance. Enable Row Level Security (RLS) policies on all tables so users can only read/write their own data unless their role permits otherwise.
- **File storage: Supabase Storage**. Profile photos are uploaded directly to a Supabase Storage bucket (e.g., `profile-photos`). The `photo_path` column in the `profiles` table stores the bucket-relative path. A public or signed URL is generated at read time — no local server storage and no third-party service (Cloudinary, S3) needed.
- **Chat implementation:** For MVP, polling (refresh every few seconds) is acceptable. Supabase Realtime (built-in) is the P2 upgrade path and requires minimal extra code — use it only if Week 3 schedule permits.

- **Photo blur:** Implement via CSS `filter: blur()` toggled in the React component based on whether a `connection` row exists between the viewer and the profile owner. No server-side image processing required.

8.2 Design Constraints

- **Design-first approach:** This semester project is evaluated heavily on Software Engineering process, which includes **design and prototyping as a critical deliverable**. The UI must be built from a proper design system — not improvised in code.
- **Figma (UI/UX Design):** All screens must be designed and prototyped in **Figma** before implementation begins. The Figma file is the source of truth for layout, spacing, colour, and component design. Antigravity/Gemini CLI should be given the Figma export or design tokens when generating frontend code.
- **Google Stitch (Design-to-Code): Google Stitch** will be used to bridge Figma designs into the React frontend. Components exported or scaffolded via Stitch must follow the Figma design system's naming conventions.
- **Design system requirements:** Define and use a consistent design system including:
  - A **colour palette** (primary, secondary, accent, neutral, error/success states).
  - A **typography scale** (heading sizes H1–H4, body, caption, label).
  - A **component library** (Button variants, Input fields, Cards, Badges, Modals, Nav bar) — all designed in Figma before being coded in React.
  - **Spacing and layout grid** (e.g., 8px base unit, defined column grid).
- **Language:** English UI. Urdu text in profile bios is acceptable and expected — ensure the font chosen supports Arabic/Urdu script or falls back gracefully.
- **Accessibility:** No formal WCAG compliance required for semester scope, but sufficient colour contrast and legible font sizes are expected to pass a visual design review.

8.3 Assumptions

- Authentication is handled entirely by **Supabase Auth** (email + password). No custom JWT logic or session management is written in Express — the Supabase client handles tokens.
- Profile photos are stored in **Supabase Storage** (`profile-photos` bucket). The `profiles` table stores only the bucket-relative `photo_path`; the full URL is constructed client-side using the Supabase Storage public URL helper. No local disk storage and no third-party image service is used.

- The "Verified" badge is **manually set by Admin** only via the admin dashboard — no automated ID verification.
- A Guardian is **linked to exactly one Candidate** for this MVP (no multi-candidate guardian management).
- A Candidate **cannot also be a Guardian** — roles are mutually exclusive and set at registration.
- Chat history is **persistent in the Supabase database** (not in-memory). All messages are rows in the `messages` table.
- There is **no email notification system** for MVP — all actions (new requests, approvals, acceptances) are visible only within the app on next login or page refresh.
- The Admin account is **seeded directly in Supabase** at deploy time (insert a row in `auth.users` + `users` table with role `admin`). No admin registration UI is needed.
- Supabase **Row Level Security (RLS)** is enabled on all tables. Policies enforce that candidates only see their own profile data, guardians only see the linked candidate's data, and admins have full read access.

Appendix A: Data Model Sketch

> For Antigravity's reference — Supabase Postgres tables. Supabase Auth manages the `auth.users` table automatically; the `users` table below **extends** it using the same `id` (UUID).
> All tables should have **Row Level Security (RLS) enabled**.

```
-- Extends Supabase auth.users. id is a UUID matching auth.users.id
users
  id           UUID PRIMARY KEY REFERENCES auth.users(id) ON DELETE CASCADE,
  name         TEXT NOT NULL,
  role         TEXT CHECK (role IN ('candidate', 'guardian', 'admin')) NOT NULL,
  is_verified  BOOLEAN DEFAULT FALSE,   -- Admin sets this manually
  is_active    BOOLEAN DEFAULT TRUE,    -- Admin can deactivate
  created_at   TIMESTAMPTZ DEFAULT NOW()

-- One profile per candidate user. Guardian users do NOT have a profiles row.
profiles
  id                     UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id                UUID UNIQUE NOT NULL REFERENCES users(id) ON DELETE
CASCADE,
  age                    INTEGER,
  city                   TEXT,
  education              TEXT,
  profession             TEXT,
  religious_values       TEXT,
  about_me               TEXT,
  photo_path             TEXT,          -- Supabase Storage bucket-relative path
                                        -- e.g. 'profile-
photos/user_id/avatar.jpg'
                                        -- Full URL constructed client-side via
                                        -- supabase.storage.from('profile-
photos').getPublicUrl(photo_path)
  partner_pref_age_min   INTEGER,
  partner_pref_age_max   INTEGER,
  partner_pref_city      TEXT,
  partner_pref_education TEXT,
  updated_at             TIMESTAMPTZ DEFAULT NOW()

-- Links one Guardian user to exactly one Candidate user (MVP: 1-to-1)
guardian_links
  id                  UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  guardian_user_id    UUID UNIQUE NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  candidate_user_id   UUID UNIQUE NOT NULL REFERENCES users(id) ON DELETE CASCADE

-- The core transaction of the platform.
-- Status flow: pending → approved_by_guardian → accepted | declined
-- If receiver has no Guardian, guardian_reviewed defaults TRUE (no gate needed)
interest_requests
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
```

```
  sender_id         UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  receiver_id       UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  status            TEXT CHECK (status IN (
                      'pending',
                      'approved_by_guardian',
                      'accepted',
                      'declined'
                    )) DEFAULT 'pending',
  guardian_reviewed BOOLEAN DEFAULT FALSE,
  created_at        TIMESTAMPTZ DEFAULT NOW(),
  updated_at        TIMESTAMPTZ DEFAULT NOW(),
  UNIQUE (sender_id, receiver_id)       -- Prevents duplicate requests

-- Created only when a Candidate accepts a guardian-approved Interest Request.
-- Unlocks chat for both users.
connections
  id           UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_a_id    UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  user_b_id    UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  connected_at TIMESTAMPTZ DEFAULT NOW(),
  UNIQUE (user_a_id, user_b_id)         -- Prevents duplicate connections

-- Chat messages. Only writable if a connection row exists for sender+receiver.
messages
  id             UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  connection_id  UUID NOT NULL REFERENCES connections(id) ON DELETE CASCADE,
  sender_id      UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  content        TEXT NOT NULL,
  sent_at        TIMESTAMPTZ DEFAULT NOW()
```

Supabase Storage

```
Bucket name : profile-photos
Access      : Private (signed URLs) or Public — choose based on photo-privacy
requirement.
              Recommended: Private bucket + signed URLs generated server-side,
              so only authenticated users with a connection can retrieve the real
photo.
              For MVP simplicity: Public bucket is acceptable; blur is enforced in
the
              React component via CSS, not URL access control.
Path format : profile-photos/{user_id}/{filename}
```

Functionality Coverage Check

Feature	Covered By
Registration & login	Supabase Auth + `users` table
Role-based routing	`users.role` field
Profile creation & edit	`profiles` table
Photo upload & display	`profiles.photo_path` + Supabase Storage
Photo blur (privacy)	React component checks `connections` table — no DB change needed
Partner preference filters	`profiles.partner_pref_*` fields
Send Interest Request	`interest_requests` (status: pending)
Guardian approval gate	`interest_requests.guardian_reviewed` + status transition
Candidate accept/decline	`interest_requests.status` → accepted/declined
Chat unlock	`connections` row created on acceptance
Chat messages	`messages` table
Guardian–Candidate link	`guardian_links` table
Admin verify user	`users.is_verified` toggle
Admin deactivate user	`users.is_active` toggle
Admin stats (totals)	COUNT queries on `users`, `interest_requests`, `connections`
Duplicate request prevention	`UNIQUE (sender_id, receiver_id)` constraint

Appendix B: Role Permission Matrix

Action	Candidate	Guardian	Admin
Register & Login	✅	✅	✅ (seeded)
Create/Edit own profile	✅	❌	❌

Action	Candidate	Guardian	Admin
Edit linked candidate's profile	❌	✅	❌
Search for matches	✅	❌	❌
Send Interest Request	✅	❌	❌
Approve/Decline (Guardian gate)	❌	✅	❌
Accept/Decline (Candidate)	✅	❌	❌
Access Chat	✅ (if connected)	❌	❌
View admin user list	❌	❌	✅
Verify / Deactivate users	❌	❌	✅
View platform stats	❌	❌	✅

End of PRD v1.1 — Rishtafy Semester Project
Changes from v1.0: Section 7 (6 metrics added), Section 8.1 (stack defined: React + Express JS + Supabase), Section 8.2 (design-first approach, Figma + Google Stitch, design system required), Section 8.3 (assumptions updated for Supabase Auth + Storage + RLS), Appendix A (full Supabase-native schema with storage, RLS notes, UNIQUE constraints, and functionality coverage check).